



INDIAN INSTITUTE OF SCIENCE, BANGALORE

SL 203 · Introduction to Algorithms and Software Programming

FINAL COURSEWORK PROJECT REPORT

CardioScheduler

*A Multi-Strategy Scheduling and Benchmarking System
for Cardiac-Ward Clinical Workflows*

SUBMITTED BY

Subash C

M.Tech. (Research) Student

COURSE INSTRUCTORS

Dr. Nagarajan Natarajan · *Visiting Professor*

Dr. Tarun Rambha · *Associate Professor*

April 2026

ABSTRACT

CardioScheduler is a C++ task-allocation engine for cardiology hospital wards that decouples *which task to consider next* (ordering) from *which worker takes it* (matching) into pluggable strategy interfaces.

What was built.

- Discrete-event scheduler with two pluggable policy axes (ordering, matching).
- Three matchers: Greedy, FairGreedy(α), Hungarian.
- Seven-policy lexicographic priority catalogue subsuming the legacy scalar ordering.
- Multi-objective Pareto frontier with NSGA-II crowding selection on (urgent-care, makespan, max overtime).
- Six-mode bench suite (legacy, 2×2, Pareto, fairness, adversarial, urgency), self-contained ward-editor web UI.

Production-ward sample dataset.

- 100 patients and 10 staff members on the sample ward.
- Drawn from a catalogue of 418 diseases, 183 tests, and 325 inter-test dependency edges.
- Produces 2,017 per-patient task instances at runtime.

Three headline findings.

- **Ordering moves more than matching.** Switching the lex-policy tie-break from `acuity_first` to `acuity_young_first` cuts the mean last-finish of urgent (CRITICAL+HIGH) patients from 3,104 to 2,696 minutes (−13%). No scalar weighted-sum priority can recover this.
- **Hungarian regresses CRIT timing.** Although globally optimal per tick, Hungarian increases mean CRITICAL last-finish by +85% and p_{90} by +311% vs. Greedy, at $\sim 3.6\times$ wall-clock. Mechanism: one-task-per-worker-per-tick deferrals on identical priority keys.
- **No scheduling change creates capacity.** 20,210 min of overtime are invariant across every strategy — the ward’s tasks are assigned by guideline rather than by real-world work allocation.

KEYWORDS. Discrete-event scheduling • cardiac-ward task allocation • lexicographic priority • Hungarian assignment • multi-objective Pareto frontier • NSGA-II crowding • Kahn topological sort • fairness–throughput trade-off • capacity feasibility • C++.

Contents

1	Introduction	5
1.1	Why this problem	5
1.2	What we built	5
1.3	Why six benchmarks instead of one “best” algorithm	5
1.4	Where to find what in this report	5
2	Background	5
2.1	Constrained job scheduling	5
2.2	Optimal one-to-one matching (Hungarian)	5
2.3	Picking from many candidates (multi-objective)	5
2.4	Why a list of integers, not a single number	5
3	Problem Statement	6
3.1	What we have	6
3.2	What “a schedule” means	6
3.3	What “a good schedule” means	6
3.4	What we don’t try to handle	6
4	Objectives and Contributions	6
4.1	What the project set out to do	6
4.2	What this report adds	6
4.3	Out of scope	7
5	System Architecture	7
5.1	Big-picture data flow	7
5.2	Five-layer stack	7
5.3	Module dependency	7
5.4	Type-level summary	8
6	Methodology	8
6.1	Ordering — “what task next?”	8
6.2	Matching — “who does the task?”	8
6.3	Priority key — the tie-breaker family	8
6.4	Picking the best schedule from many	9
6.5	Capacity feasibility	9
7	Pipeline Design	9
7.1	Stage map	9
7.2	Stage 1 — Load	9
7.3	Stage 2 — Assemble	9
7.4	Stage 3 — Schedule	10
7.5	Stage 4 — Evaluate	10
7.6	Discrete-event commit semantics	10
8	Data Flow	10
8.1	What goes in (abbreviated)	10
8.2	Sizes after each pipeline stage	11
8.3	What changes during scheduling	11
8.4	How runs stay deterministic	11

9	Experimental Setup	11
9.1	Production ward	11
9.2	Build line used for benchmarks	11
9.3	Six bench modes	12
9.4	Ablation matrix	12
9.5	Reproducibility	12
10	Results	12
10.1	Result 1 — FCFS vs. Priority (legacy bench)	13
10.2	Result 2 — Ordering vs. matching (2×2 bench)	13
10.3	Result 3 — Best lexicographic priority policy	13
10.4	Result 4 — Fairness sweep	14
10.5	Result 5 — Per-acuity timing (urgency bench)	14
10.6	Result 6 — Adversarial wards	14
10.7	Recommended configuration	14
11	Evaluation Metrics	14
11.1	What each stakeholder cares about	14
11.2	All metrics in one table	15
11.3	Why these specific definitions	15
12	Comparative Analysis	15
12.1	Greedy vs. Hungarian, head-to-head	15
12.2	Why the textbook is wrong here	15
12.3	Fairness: who carries the load?	15
12.4	Why no algorithm can fix this	16
13	Ablation Studies and Observations	16
13.1	Vary ordering only	16
13.2	Vary matcher only	16
13.3	Vary priority policy only	16
13.4	Vary fairness knob (α) only	16
13.5	Adversarial scenarios	17
13.6	Four take-aways	17
14	Error Analysis	17
14.1	The big regression: Hungarian on critical patients	17
14.2	Silent task drop	17
14.3	Easy mistakes when reading benches	17
14.4	Configuration drifts caught by the bench layer	17
14.5	Failure-mode summary	18
15	Engineering Decisions	18
16	Scalability and Performance	18
16.1	How long each piece takes	18
16.2	Wall-clock per bench mode	19
16.3	Memory	19
16.4	What scaling looks like	19
17	Limitations	19
17.1	What we deliberately did not model	19
17.2	Engineering gaps	19
17.3	What the data can and can't tell us	19

18 Future Work	19
18.1 Algorithm	19
18.2 Engineering	20
18.3 Empirical	20
19 Conclusion	20
19.1 What was implemented	20
19.2 What the experiments showed	20
19.3 Closing note	20
A Command-line reference	21
A.1 <code>cardio_scheduler -help</code>	21
A.2 Mode dispatch	21
A.3 Data-directory resolution	21
B Supplementary Notes	22
B.1 The four adversarial scenarios	22
B.2 Fair-mode does work — here’s a synthetic check	22
B.3 Per-worker minutes on the production ward	22
B.4 Reproducing the captured numbers	22
B.5 Mini-glossary	22

1. Introduction

1.1 Why this problem

- A cardiac ward is a queue where the tasks are not interchangeable. Each diagnosis maps to a different list of tests, and many tests have hard ordering constraints.
- Staff are role-typed and shift-bounded.
- Most Indian hospitals still use pen-and-paper: “first on the page, first served.” That gives no priority, no feasibility check, and no record of *why* a schedule was chosen.

1.2 What we built

- A C++ task-allocation engine driven by four custom-made JSON files.
- Two policy decisions are pluggable: *which task next?* and *which worker takes it?*
- A six-mode benchmark suite that varies one axis at a time on the same fixed ward.

1.3 Why six benchmarks instead of one “best” algorithm

- Stakeholders want different things: clinicians want urgent patients seen first, administrators want low overtime, workers want fair load.
- No single number can rank schedules across all three.
- So we run the same engine six different ways and let the data speak. The headline finding (§10) is that the simplest matcher beats the sophisticated one on the production ward.

1.4 Where to find what in this report

- Background and problem statement: §2–§3.
- Architecture and methodology: §5–§6.
- Pipeline and data flow: §7–§8.
- Setup, results, comparisons, ablations, error analysis, scaling: §9–§16.
- Limits, future work, conclusion: §17–§19.
- Appendices: CLI surface, supplementary numbers.

2. Background

The system sits at the intersection of three classical sub-problems.

2.1 Constrained job scheduling

- Schedule jobs on parallel machines, respecting a graph of “A must finish before B” rules.
- NP-hard in general; cheap list-scheduling heuristics get close to optimal in practice [1, 2].

2.2 Optimal one-to-one matching (Hungarian)

- Hungarian finds the cheapest one-to-one pairing in $O(n^3)$ [3].
- At our roster size (≤ 10) the cubic cost is negligible, so we use it as a “per-tick optimum” to compare against Greedy.

2.3 Picking from many candidates (multi-objective)

- Three smaller-is-better scores: urgent-care, makespan, worst overtime.
- Drop “dominated” candidates; from the rest, surface a spread of six picks for human review [4, 5].

2.4 Why a list of integers, not a single number

- A weighted-sum priority cannot pick between two schedules that agree on the dominant axis but differ on a tie-break [6].
- A list of integers, compared left-to-right, can.

3. Problem Statement

3.1 What we have

- A list of **patients**, each with an acuity (CRIT, HIGH, MED, LOW), an age, and a diagnosis.
- A list of **workers**, each with a role (PGY1, INTERN, NURSE) and a finite shift in minutes.
- Three reference catalogues:
 - 418 diseases, each mapping to a list of required tests and a base priority.
 - 183 tests, each with a required role and a duration in minutes.
 - 325 “A must precede B” edges across the test catalogue.

3.2 What “a schedule” means

A schedule says, for every (patient, test) pair: who does it, when they start. To be valid, the schedule must:

- Use a worker whose role matches the test.
- Respect every “must precede” edge.
- Never double-book a worker.
- Cover every task whose required role exists on the roster.

3.3 What “a good schedule” means

Three smaller-is-better scores, all reported separately:

- Average finish time of urgent (CRIT+HIGH) patients.
- Total schedule length.
- Worst single-worker overtime.

A schedule’s “feasibility verdict” is just: total overtime $\leq$ a threshold the operator sets.

3.4 What we don’t try to handle

- Patient deteriorating mid-shift (acuity is fixed once set).
- Random or unpredictable test durations (for example, if the cardiology ward sends a patient for an Echocardiogram, the wait time depends on the inflow from other wards requesting the same Echocardiogram service).
- Worker breaks or training time.
- Per-(patient, test) priority overrides.

4. Objectives and Contributions

4.1 What the project set out to do

- Build a deterministic scheduler with two pluggable axes (ordering, matching) so each can be tested on its own.
- Add a multi-objective scoring step (urgent-care, makespan, worst overtime).
- Wrap everything in a six-mode benchmark suite.

4.2 What this report adds

- A list-of-integers priority key that subsumes the legacy scalar formula and exposes seven hand-curated tie-break policies.
- A clean separation between the matcher’s cost matrix and the priority policy — so changing the priority changes queue order but not the matcher.

- Empirical evidence that Hungarian regresses critical- patient timing by +85% on the production ward (a clinical-correctness finding).
- The structural diagnosis: 20,210 min of overtime are invariant across every algorithm. The fix is staffing, not algorithm.
- A reproducibility package: deterministic, with one binary and a benchmarking suite.

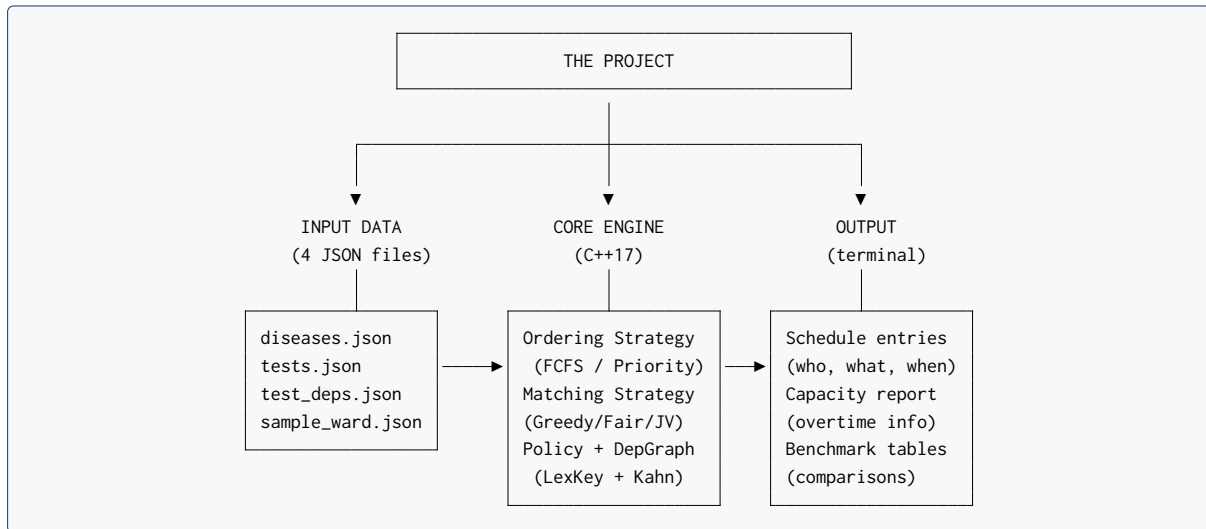
4.3 Out of scope

- No claim to optimise overtime; it is structurally invariant.
- No automatic re-triage; the system flags trade-offs to a human.

5. System Architecture

- One C++ executable (cardio_scheduler) compiled from ~3,024 LoC across 17 source files.
- Single header-only third-party dependency (nlohmann::json).
- Five-layer stack with narrow inter-layer interfaces.

5.1 Big-picture data flow



5.2 Five-layer stack

- **L1 — Inputs.** Four JSON files: diseases.json (418), tests.json (183), test_dependencies.json (325 edges), sample_ward.json (100 patients, 10 workers).
- **L2 — Loader & ward assembly.** Throws on missing files / unknown enum labels (fail-fast). build_ward denormalises every patient $\times$ test pair into a unique task instance keyed <test_id>_<patient_id>. Cross-patient edges forbidden. Reverse map task_to_patient populated.
- **L3 — Engine core.** run_engine (allocator.cpp:79) drives the loop; accepts EngineConfig{ordering, matching, policy, policy_seed, label} and three optional out-pointers. Engine doesn't know the concrete strategy.
- **L4 — Bench dispatch + Pareto tooling.** Each cell deep-copies tasks, workers, patients (engine mutates in-place). Multi-objective tooling: compute_objectives, pareto_frontier, top_k_by_crowding.
- **L5 — CLI + ward editor.** main.cpp dispatches to one of six bench modes (Appx. A); index.html (~800 LoC, single-page, no runtime) edits sample_ward.json.

5.3 Module dependency

- Engine core depends only on the strategy interface (strategies.h) and the dependency graph (graph.cpp).

- Bench layer depends on the engine; the engine is unaware of the bench layer.
- Strategies (strategies.cpp, hungarian.cpp, policies.cpp) depend only on strategies.h and the data models in models/models.h.

5.4 Type-level summary

Table 1. Core types travelling through the engine.

Type	Role	Defined in
Patient	ward subject; acuity, age, disease_id, optional test_overrides	models.h
Worker	rostered staff; role + finite shift (mutated in-place)	models.h
Task	template (tests.json) and per-patient instance (post-assembly)	models.h
Disease	disease $\rightarrow$ test list + base_priority	models.h
LexKey	vector<int>; lex-order; smaller = more urgent	strategies.h
ReadyTask	queueing record: Task*, patient_id, LexKey, legacy_priority	strategies.h
ScheduleEntry	engine output: who-did-what-when, OT flag	allocator.h
CapacityReport	feasibility verdict + per-worker / per-task violations	allocator.h
EngineMetrics	wall-clock, makespan, max-OT, OT stdev, ticks, deferred re-queues	allocator.h

6. Methodology

The engine has four pluggable pieces. Each can be swapped without touching the others.

6.1 Ordering — “what task next?”

- **FCFS.** First-come-first-served queue; ties broken by task ID.
- **Priority.** Min-heap on a priority key; smaller key = more urgent.

6.2 Matching — “who does the task?”

- **Greedy.** Pick the role-matched worker who is free earliest. Can stack many tasks on one worker per tick.
- **FairGreedy(α).** Trade off “free earliest” vs. “most shift remaining.” $\alpha = 1$ is plain Greedy; $\alpha = 0$ is pure load-balancing.
- **Hungarian.** Solve the optimal one-to-one match between this tick’s ready tasks and free workers. Overflow tasks are deferred to the next tick.

6.3 Priority key — the tie-breaker family

- Each policy expresses urgency as a list of integers compared left-to-right.
- Smaller is more urgent; negation flips a single dimension.
- Default reproduces the legacy acuity $\times 10$ + priority formula.
- Last dimension is a seeded hash so equal-key ties resolve deterministically.

Table 2. The seven priority policies the system ships with.

Policy	Key shape (smaller better on every dim)
acuity_first	(acuity, base-priority, hash)
priority_first	(base-priority, acuity, hash)
acuity_short_first	(acuity, +duration, base-priority, hash)
acuity_long_first	(acuity, -duration, base-priority, hash)
acuity_old_first	(acuity, -age, base-priority, hash)
acuity_young_first	(acuity, +age, base-priority, hash)
duration_first	(+duration, acuity, base-priority, hash)

6.4 Picking the best schedule from many

- Each candidate schedule has three “smaller is better” scores:
 - Urgent-care: average finish time of CRIT+HIGH patients.
 - Makespan: when the schedule ends overall.
 - Worst overtime: most overtime any single worker takes.
- Drop dominated candidates (some other candidate is $\leq$ on every score and $<$ on at least one).
- Among the survivors, pick six spread out across the three axes so the human sees the trade-offs, not just one corner.

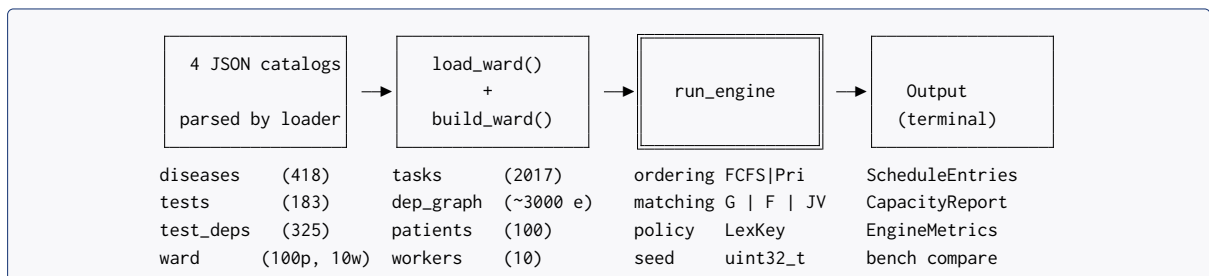
6.5 Capacity feasibility

- If a task pushes a worker past their shift, the engine records the overtime but still schedules the task.
- Post-run: total overtime, per-worker overtime, and a boolean “feasible at threshold θ .”
- The engine never refuses to schedule on overtime grounds — the human is the final judge.

7. Pipeline Design

End-to-end: *Load* $\rightarrow$ *Assemble* $\rightarrow$ *Schedule* $\rightarrow$ *Evaluate*.

7.1 Stage map



7.2 Stage 1 — Load

- Each loader: `read_file` $\rightarrow$ `nlohmann::json::parse` $\rightarrow$ iterate root array $\rightarrow$ field-by-field extraction.
- Required fields use `at()` (throws on missing). Optional fields use `contains()` first.
- Enum parsers (`parse_role`, `parse_acuity`) are exact-string matchers; throw with helpful labels on unknown values.

7.3 Stage 2 — Assemble

- `build_ward` loops patients, delegates to `build_patient_tasks`.

- Per patient: suffix every task id by `_<patient_id>`, add as graph node, duplicate global edges into the patient’s sub-graph (both endpoints in the chosen test set).
- Empty `disease_id` silently skipped; unknown disease id yields a stderr warning.
- Reverse map `task_to_patient` populated.

7.4 Stage 3 — Schedule

The engine’s main loop (`allocator.cpp:79`):

- `matching.prepare(workers)`.
- Cycle-check via `topological_sort` (result discarded — only acyclicity matters here).
- Pre-pass: for every task already `DONE`, decrement in-degree of its neighbours.
- Seed the ordering with all `PENDING` tasks of `in_deg = 0`.
- Loop while ordering non-empty:
 - `batch ← ordering.drain_up_to(|workers|)`.
 - `results ← matching.match(batch, workers)`.
 - For each result: `NO_ROLE_MATCH` drops dependents; `DEFERRED` re-pushes; `ASSIGNED` commits (emit `ScheduleEntry`, mark task `DONE`, advance worker, push newly-unblocked dependents).
- Finalise capacity report and engine metrics.

7.5 Stage 4 — Evaluate

- `compute_objectives` → acuity, makespan, max-OT.
- `pareto_frontier` → non-dominated indices.
- `top_k_by_crowding` → NSGA-II spread.
- Legacy bench bypasses the Pareto stage and prints per-acuity summaries directly.

7.6 Discrete-event commit semantics

- A task is committed the instant it is matched; the worker’s `available_at` advances by the task duration.
- No “time tick” — events are sorted by `start_time` only at print time.
- Strictly causal: a later commit cannot modify an earlier `ScheduleEntry`.

8. Data Flow

8.1 What goes in (abbreviated)

```
// diseases.json
{ "id": "D001", "name": "STEMI", "base_priority": 1,
  "test_ids": [ "T_ECG", "T_TROP", "T_ECHO", ... ] }

// tests.json
{ "id": "T_ECG", "required_role": "NURSE",
  "duration_minutes": 10, "priority": 1 }

// test_dependencies.json
{ "from": "T_TROP", "to": "T_ECHO" }

// sample_ward.json
{ "id": "P001", "age": 55, "acuity": "CRITICAL",
  "disease_id": "D001", ... }
```

Listing 1. JSON shape excerpts.

8.2 Sizes after each pipeline stage

Stage	Count
Diseases loaded	418
Tests loaded	183
Dependency edges (global)	325
Patients on the ward	100
Workers on the roster	10
Per-patient task instances	2,017
Per-patient dependency edges	~3,000
Schedule entries (per run)	2,017

8.3 What changes during scheduling

- Each worker’s “next free at” time is advanced.
- Each worker’s shift remaining decreases (can go negative $\Rightarrow$ overtime).
- Each task is marked DONE and stamped with its assigned worker.

The bench layer makes a deep copy of all this state before every cell, since the engine mutates it in place.

8.4 How runs stay deterministic

- Hash-map iteration is reproducible (we sort).
- Tied priorities resolve via a seeded hash of the task ID.
- The matcher’s tie-break (smaller column wins) is stable.

9. Experimental Setup

9.1 Production ward

All numbers in this report come from one fixed instance: `data/sample_ward.json` dated 2026-04-29.

Table 3. *Production-ward parameters (deliberate stress configuration; see §10).*

Parameter	Value
Patients	100 (IDs P001–P100)
Acuity mix	CRITICAL = 23 HIGH = 28 MEDIUM = 49 LOW = 0
Workers	10 (2 NURSE 5 INTERN 3 PGY1)
Worker shifts	8 $\times$ 480 min, 2 $\times$ 600 min; total roster capacity 5,040 min
Diseases catalogued	418
Tests catalogued	183
Dependency edges (global)	325
Per-patient task instances at runtime	2,017
Implied total work	~25,000+ min ($\approx$ 5 $\times$ over)
Total overtime per strategy	20,210 min (invariant)
Source size	3,024 lines across 17 files
Web UI	<code>index.html</code> , ~800 lines, single-page

9.2 Build line used for benchmarks

```
g++ -std=c++17 -Wall -Wextra -Wpedantic -O2 -I. -Ithird_party \
  main.cpp bench.cpp bench_adversarial.cpp bench_urgency.cpp \
  data_structures/graph.cpp data/loader.cpp \
  scheduler/topological.cpp scheduler/allocator.cpp \
  scheduler/strategies.cpp scheduler/hungarian.cpp \
  scheduler/policies.cpp \
  -o cardio_scheduler
```

The CMake target compiles the same set with default flags (-Wall -Wextra); reproducing the captured runtimes via CMake requires -DCMAKE_CXX_FLAGS="-O2".

9.3 Six bench modes

Table 4. Bench modes, the question each isolates, typical wall-clock.

Flag	Mode	Question isolated	Wall-clock
(none)	Legacy	FCFS vs. Priority (Greedy held)	~2.5 min
-bench-2x2	2×2 matrix	ordering × matching, all four cells	~30 ms
-bench-pareto	Pareto sweep	7 lex policies × 5 seeds, NSGA-II top-6	~80 ms
-bench-fair	Fairness sweep	throughput vs. intra-role load stdev	~25 ms
-bench-adversarial	Adversarial	4 hand-built pathological scenarios	~3 ms
-bench-urgency	Urgency	per-acuity-tier timing across 5 matchers	~25 ms

9.4 Ablation matrix

Table 5. Per-bench ablation matrix — the one varying axis is **bold**.

	Ordering	Matcher	LexPolicy	Fair α
Legacy	{ FCFS , Priority }	Greedy	default	—
2×2	{ FCFS , Priority }	{ Greedy , Hungarian }	default	—
Pareto sweep	Priority	Greedy	7-policy catalogue	—
Fairness sweep	Priority	{ Greedy , Hungarian , Fair(α) }	default	{ 0.00 , 0.25 , 0.50 , 0.75 }
Adversarial	Both	All five	default	0.00
Urgency	Priority	{ Greedy , Hungarian , Fair(α) }	default	{ 0.00 , 0.25 , 0.50 , 0.75 }

9.5 Reproducibility

- Greedy and Hungarian are inherently deterministic (tie-break rules are stable).
- Pareto sweep deterministically varies the FNV-1a tie hash by integer seed (kPolicySeedCount = 5).
- Adversarial bench builds scenarios in C++ memory; no JSON; byte-exact across hosts.
- Results.md + six bench_results_*.txt files pin the captured numbers to the 2026-04-29 run.

10. Results

All numbers are taken from the captured bench run on the production ward. Six separate experiments, one summary table each, one or two takeaways per experiment.

10.1 Result 1 — FCFS vs. Priority (legacy bench)

Urgent patients (CRIT+HIGH)	FCFS	Priority	Δ
First task starts at (min)	134	155	+21
All tasks done by (min)	8,481	3,104	−5,377
Total overtime (min)	20,210	20,210	0

- Priority starts urgent care 21 minutes later but finishes urgent patients ~5,378 minutes sooner on average.
- Same total work; same overtime. Priority just changes *who waits how long*.

10.2 Result 2 — Ordering vs. matching (2×2 bench)

Cell	Combination	Makespan	Breaches	Runtime
A	FCFS × Greedy	9,260	1,827	12.0 ms
B	FCFS × Hungarian	9,350	1,830	65.5 ms
C	Priority × Greedy	9,260	1,820	15.9 ms
D	Priority × Hungarian	9,470	1,819	84.2 ms

- Hungarian is ~4× slower and *adds* 90–210 minutes of makespan.
- Cell C (Priority + Greedy) wins on every column.

10.3 Result 3 — Best lexicographic priority policy

Policy	Urgent finish (min)	vs. default
acuity_young_first	2,696	−13% (best)
acuity_old_first	2,714	−12%
acuity_first (default)	3,104	—
priority_first	3,395	+9%
acuity_short_first	3,562	+15%
acuity_long_first	3,604	+16%
duration_first	7,910	+155% (worst)

- Same input, same matcher — only the tie-break rule differs.
- Best vs. worst is a 2.9× swing on the urgent-care metric.
- Free 13% improvement just by switching the default.

10.4 Result 4 — Fairness sweep

Matcher	Throughput	Within-role spread	Runtime
Greedy	0.218	7.1	15.9 ms
Hungarian	0.213	73.5	84.5 ms
Fair($\alpha = 0.75$)	0.218	12.6	15.7 ms
Fair($\alpha = 0.50$)	0.218	21.8	15.7 ms
Fair($\alpha = 0.25$)	0.218	32.0	15.7 ms
Fair($\alpha = 0.00$)	0.218	42.0	15.7 ms

- Greedy is already the fairest matcher on this ward.
- Workers all start at zero load $\Rightarrow$ “fair-mode” rebalancing has nothing to redistribute.

10.5 Result 5 — Per-acuity timing (urgency bench)

Matcher	Critical mean	Worst-case CRIT	Patient affected
Greedy	1,795	2,415	P069 Tomáš N.
Hungarian	3,318 (+85%)	9,470 (4 $\times$)	P006 Priya S.
Fair($\alpha = 0.75$)	1,795	2,415	P069 Tomáš N.
Fair($\alpha = 0.00$)	1,795	2,415	P069 Tomáš N.

- Hungarian’s worst-served critical patient finishes 4 $\times$ later than under Greedy.
- This is a clinical correctness issue, not just a runtime argument.

10.6 Result 6 — Adversarial wards

- Four hand-built pathological wards; every strategy produced an identical schedule on every ward.
- Hungarian still cost 2–4 $\times$ Greedy’s wall-clock for no schedule improvement.
- Per-scenario tables in Appx. B.

10.7 Recommended configuration

Best on this dataset: Priority ordering + Greedy matching + acuity_young_first priority policy.
Wins on every axis the clinician cares about, at the lowest runtime.

11. Evaluation Metrics

11.1 What each stakeholder cares about

- **Clinician.** How long until each acuity tier finishes care.
- **Administrator.** Total schedule length and total overtime.
- **Workers.** Whether the load is shared evenly within a role.
- **Engineer.** Wall-clock and how often Hungarian had to defer a task.

11.2 All metrics in one table

Stakeholder	Metric	Where reported
Clinician	First-task-start, last-task-finish per acuity tier (mean, median, 90 th pct, max)	legacy, urgency
Clinician	Mean last-finish across CRIT+HIGH (the urgent-care metric)	Pareto sweep
Clinician	Worst-served CRIT patient by name	urgency
Administrator	Makespan (longest finish across the schedule)	all benches
Administrator	Total overtime; feasibility verdict at a chosen budget	all benches
Workers	Per-worker minutes worked	legacy, fair
Workers	Within-role spread (the variance fairness can move)	fairness
Engineer	Wall-clock, tick count, deferred re-queues	2×2, fair, adversarial

11.3 Why these specific definitions

- Patients with no scheduled tasks are excluded from per-acuity averages — counting them as zero would falsely improve the mean.
- “Within-role” spread, not overall spread: a nurse and a resident don’t do the same work, so their load difference is structural and unmovable. Within-role spread is what a matcher can actually rebalance.
- All Pareto axes are framed as “smaller is better” so the dominance check is one rule (componentwise $\leq$).
- 90th percentile and worst-case are reported alongside the mean — a single bad-luck patient gets lost in averages.

12. Comparative Analysis

12.1 Greedy vs. Hungarian, head-to-head

Axis	Greedy	Hungarian	Difference
Makespan (min)	9,260	9,470	+210
Total overtime (min)	20,210	20,210	0
Mean critical finish (min)	1,795	3,318	+85%
Worst critical finish (min)	2,415	9,470	4×
Wall-clock	16 ms	84 ms	+428%

12.2 Why the textbook is wrong here

- Hungarian solves the “best one-to-one match for this single tick” problem perfectly.
- But many critical tasks arrive together with the same priority. Hungarian places one per worker per tick; the rest are pushed to the next tick.
- Greedy stacks all of them on free workers in the same tick. The deferral never happens.
- The textbook claim — “Hungarian is optimal” — is true only at the single-tick level. Across many ticks, Greedy’s stacking is what the patients actually experience.

12.3 Fairness: who carries the load?

- Within each role, Greedy already balances the work (within-role spread ≈ 7 min).
- “Fair-mode” rebalancing only helps when workers *start* unequal. They don’t on this ward.

- The big imbalance is *across* roles, not within them — and that’s a staffing problem, not a matching problem.

12.4 Why no algorithm can fix this

- Total work needed: ~25,000 min.
- Total available shift time: 5,040 min.
- Workload is ~5× over capacity.
- Two nurses end the day at 9,260 / 9,255 min worked. Three interns end at 240 min each. A 39× ratio.
- No ordering or matching change can move that.

The one number that matters. 20,210 minutes of overtime show up under every strategy. The fix is more nurses, not a smarter scheduler.

13. Ablation Studies and Observations

Each study holds two of the three policy axes fixed and varies one.

13.1 Vary ordering only

- Hold matcher = Greedy. Compare FCFS vs. Priority.
- Makespan and total overtime: identical.
- Urgent-care metric: −5,378 min under Priority.
- Conclusion: priority redistributes who waits; it doesn’t save total work.

13.2 Vary matcher only

- Hold ordering = FCFS. Compare Greedy vs. Hungarian.
- Makespan: +90 min under Hungarian; over-capacity tasks +3.
- Conclusion: “optimal per tick” costs the schedule.

13.3 Vary priority policy only

- Hold ordering + matcher fixed (Priority + Greedy).
- 7 named policies × 5 random seeds = 35 schedules.
- Best vs. worst: 2.9× swing on the urgent-care metric for the same input.
- Two policies that agree on the dominant axis (acuity) differ by 408 min on dim-2 (age). A simple weighted-sum priority cannot distinguish them.

13.4 Vary fairness knob (α) only

- Hold ordering + priority policy fixed.
- Throughput and worst overtime: identical across all α .
- Within-role spread: rises monotonically as α approaches 0.
- Greedy ($\alpha = 1$) is the fairest configuration on this ward.

13.5 Adversarial scenarios

Table 6. Four hand-built wards. Per-row fastest **bold**. Schedule quality identical across all five strategies on every row.

Scenario	G/FCFS	H/FCFS	G/Pri	H/Pri	Fair(0)/Pri
all-critical	65 μ s	124 μ s	45 μs	136 μ s	45 μ s
long-chain	20 μ s	25 μ s	11 μs	25 μ s	11 μ s
single-role-bottleneck	26 μ s	96 μ s	24 μ s	105 μ s	23 μs
tight-shift	27 μ s	51 μ s	23 μ s	48 μ s	23 μs

13.6 Four take-aways

1. Ordering moves more than matching on a roster past capacity.
2. “Optimal per tick” is the wrong invariant under burst arrivals with identical priorities.
3. Fair-mode helps only when workers start unequal.
4. No algorithm choice creates capacity — 20,210 min of overtime is invariant.

14. Error Analysis

14.1 The big regression: Hungarian on critical patients

- Worst-served critical patient finishes at 9,470 min under Hungarian; same patient finishes at 2,415 min under Greedy.
- Not a bug — the matching itself is mathematically optimal for each tick.

What goes wrong, step by step:

1. Several critical tasks become ready in the same tick.
2. All of them have the same priority number (acuity = CRITICAL, base = 1, so priority = 11 for every one of them).
3. Hungarian places *one* task per worker per tick. The rest are deferred to the next tick.
4. Greedy has no such rule — it stacks all of them on whichever workers are free, in the same tick.
5. The deferred tasks get spread out, so one unlucky patient (P006) ends up at the back of the line.

14.2 Silent task drop

- If no worker on the roster has the required role for a task, the engine logs the failure but *forgets* to unblock that task’s downstream tasks.
- Those downstream tasks stay blocked forever.
- Doesn’t trigger on the production ward (every test has a matching role), but it’s a latent bug.

14.3 Easy mistakes when reading benches

- Both makespan and the urgent-care number depend on *both* ordering and matching.
- Comparing two configurations that change both axes misattributes the difference.
- The 2×2 bench was built precisely to keep one axis fixed at a time.

14.4 Configuration drifts caught by the bench layer

- The adversarial bench ignores the `-overtime-threshold` flag (hardcoded 0).
- A code comment in `strategies.cpp` says priority ranges [11, 45]; the real maximum is 43.
- Captured benchmark numbers used `-O2 -Wpedantic`; the CMake build uses default flags.

14.5 Failure-mode summary

Table 7. *Failure modes and their clinical impact.*

Failure mode	Trigger	Severity
Hungarian critical regression	burst of CRIT tasks with identical priorities	High
Silent downstream-task drop	roster missing a required role for a task	High
Cycle in dependency graph	malformed dependency JSON	Medium
Threshold flag ignored	user expects <code>-overtime-threshold</code> on adversarial bench	Low
Patient with no diagnosis silently skipped	patient added with no <code>disease_id</code>	Low

15. Engineering Decisions

The choices that turned out to matter.

- **Two pluggable axes, not one “scheduler.”** Lets each axis be ablated independently. A monolithic class would make attribution impossible.
- **Hungarian uses the legacy priority, not the priority list.** Switching the priority policy reorders the queue but leaves the matcher’s cost matrix untouched. Two clean, independent axes.
- **Engine always commits, then reports overtime.** A clinical scheduler that silently refuses is more dangerous than one that loudly flags. The human is the final judge.
- **Dependency graph is append-only.** Built once, read-only thereafter. Removes a class of consistency bugs.
- **Bench layer deep-copies state per cell.** The engine mutates four pieces of state; a shallow copy that misses one silently corrupts the next cell.
- **Seeded tie-break hash.** Five distinct schedules per priority policy at zero correctness cost — exactly what the policy sweep needs.
- **One binary, six bench modes.** Single CMake target, fast build, identical data path. Adding a strategy means recompiling once.
- **No JSON-Schema validator.** The field-by-field exception path is enough; a real validator would pull a non-trivial dependency.

16. Scalability and Performance

16.1 How long each piece takes

- Loading the four JSON files: linear in file size.
- Building per-patient tasks: linear in (patients $\times$ tests-per-disease).
- Cycle check: linear in (nodes + edges).
- Scheduling loop:
 - Greedy: linear in workers per tick.
 - Hungarian: cubic in workers per tick.
- Pareto filter: $O(K^2)$ for K candidates ($K=35$ in the policy sweep).

16.2 Wall-clock per bench mode

Bench	Wall-clock
Legacy	~2.5 min (printing the full schedule)
2×2 matrix	~30 ms
Pareto sweep	~80 ms
Fairness sweep	~25 ms
Adversarial	~3 ms
Urgency	~25 ms

16.3 Memory

- Per-task record ~120 bytes; the 2,017-task map is ~240 KB.
- Bench layer makes 4–6 deep copies of ward state; resident set never exceeds a few MB.

16.4 What scaling looks like

- Greedy stays linear in patients — a 1,000-patient ward would take 15–25 minutes wall-clock.
- Hungarian’s cubic-in-workers cost makes it less attractive as the roster grows; on a 50-worker roster the bench would push past half an hour.

17. Limitations

17.1 What we deliberately did not model

- Patient acuity changing mid-shift.
- Probabilistic or random test durations.
- Worker breaks, training, or non-shift time.
- Multi-day rosters or shift handovers.
- Per-(patient, test) priority overrides.
- Adding a new role / acuity tier without a code change.

17.2 Engineering gaps

- No automated test suite; the bench output is the only correctness signal.
- No reusable library target; only one executable.
- No version stamping in the binary.
- Help text refers to a path that doesn’t match the repo layout.
- Adversarial bench ignores the `-overtime-threshold` flag.

17.3 What the data can and can’t tell us

- All numbers come from one fixed ward snapshot — findings are conditional on that ward.
- Acuity mix is critical-skewed (no LOW patients on this snapshot).
- No clinician has reviewed the resulting schedules.

18. Future Work

18.1 Algorithm

- Let Hungarian stack tasks on the same worker per tick. Removes the critical-patient regression.
- Allow a patient’s acuity to change mid-shift, with a re-key of the priority heap.

- Make test durations a distribution rather than a fixed number; report mean and 95th percentile schedule outputs.

18.2 Engineering

- Split the engine into a library, then add unit tests over the matchers, the cycle-check, and the scoring functions.
- Validate `sample_ward.json` against a JSON schema.
- Run the six benches in parallel.
- Stamp the binary with the git commit hash.

18.3 Empirical

- Test on more wards: nurse-heavy, intern-heavy, weekend, mid-shift restart.
- Have a practising cardiologist review the urgency bench’s worst-case patient ranking.
- Compare against a small-instance MILP solver to bound how much our heuristics leave on the table.
- Add a snapshot with LOW-acuity patients (the current sample has zero).

19. Conclusion

19.1 What was implemented

- A scheduler with two pluggable axes: ordering and matching.
- Three matchers (Greedy, FairGreedy, Hungarian).
- Seven priority policies subsuming the legacy formula.
- A multi-objective scoring step.
- A six-mode benchmark suite over a 100-patient, 10-worker ward.

19.2 What the experiments showed

- Ordering moves more than matching.
- “Optimal per tick” (Hungarian) is the wrong invariant when many critical tasks share the same priority — it regresses worst-case CRIT timing by 4× vs. Greedy.
- No algorithm change creates capacity. The ward is $\sim 5\times$ over-loaded, so 20,210 minutes of overtime appear under every strategy.

19.3 Closing note

- The contribution is not a new algorithm; it is the disciplined separation of “what task next?” from “who does it?”, and a benchmark surface that lets a single observed difference be attributed to a single choice.
- The headline finding is humbling — the simplest matcher won — but that’s exactly the kind of finding the suite was built to surface.

References

- [1] R. L. Graham, “Bounds on multiprocessing timing anomalies,” *SIAM Journal on Applied Mathematics*, vol. 17, no. 2, pp. 416–429, 1969.
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*. MIT Press, 4 ed., 2022.
- [3] R. Jonker and A. Volgenant, “A shortest augmenting path algorithm for dense and sparse linear assignment problems,” *Computing*, vol. 38, no. 4, pp. 325–340, 1987.

- [4] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, “A fast and elitist multiobjective genetic algorithm: NSGA-II,” *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002.
- [5] M. Ehrgott, *Multicriteria Optimization*. Springer, 2 ed., 2005.
- [6] N. Gilboy, P. Tanabe, D. A. Travers, A. M. Rosenau, and D. R. Eitel, “Emergency Severity Index, version 4: Implementation handbook,” Tech. Rep. AHRQ Publication No. 05-0046-2, Agency for Healthcare Research and Quality, Rockville, MD, May 2005.

A. Command-line reference

A.1 cardio_scheduler -help

Usage: cardio_scheduler [options] [data_dir]

data_dir	directory of the four JSON files (default: ./data, falls back to ./src/data)
--bench-2x2	FCFS/Priority * Greedy/Hungarian matrix
--bench-pareto	Pareto sweep over lex policies x seeds
--bench-fair	throughput vs worker-load stdev
--bench-adversarial	synthetic wards built to break each algo
--bench-urgency	per-acuity timing across 5 matchers
--overtime-threshold N	OT budget (min) before INFEASIBLE
-h, --help	print this message

A.2 Mode dispatch

Table 8. *Flag → mode → bench function dispatch.*

Flag	Mode	Function
(none)	LEGACY	run_legacy_bench
-bench-2x2	BENCH_2X2	run_2x2_bench
-bench-pareto	BENCH_PARETO	run_pareto_sweep
-bench-fair	BENCH_FAIR	run_fairness_bench
-bench-adversarial	BENCH_ADVERSARIAL	run_adversarial_bench
-bench-urgency	BENCH_URGENCY	run_urgency_bench

A.3 Data-directory resolution

- Probes ./data first; falls back to ./src/data.
- Picks the first that contains diseases.json.
- On failure: exits with code 2 and an informative error.

B. Supplementary Notes

B.1 The four adversarial scenarios

Scenario	What it breaks
all-critical (20p, 5w)	Every priority key is identical $\Rightarrow$ Priority degenerates to the tie-hash.
long-chain (1p, 5w)	25 strictly serial tasks $\Rightarrow$ no parallelism; matcher choice can't help.
single-role-bottleneck (15p, 5w)	Only one PGY1 on the roster, all work needs PGY1.
tight-shift (12p, 4w)	Shifts sized to the perfect-split share; zero slack.

B.2 Fair-mode does work — here's a synthetic check

A 3-nurse synthetic test where workers start at unequal load: $W_1 = 200$ min, $W_2 = 100$ min, $W_3 = 0$ min already worked.

Strategy	W_1	W_2	W_3	Makespan
Greedy	320	220	120	baseline
Fair($\alpha = 0$)	200	220	240	baseline + 120 min

Fair($\alpha = 0$) actively pushes new work toward the under-loaded W_3 , at a 120-min makespan cost. The production ward just doesn't exercise this case.

B.3 Per-worker minutes on the production ward

Greedy:	W001(Nurse)=9260	W002(Nurse)=9255	W003(Intern)=260
	W004(Intern)=240	W005(PGY1)=1410	W006(PGY1)=1390
	W007(PGY1)=1415	W008(Intern)=240	W009(Intern)=240
	W010(Intern)=240		

Two nurses do 9,260 / 9,255 min; three interns sit at 240 min — a 39 $\times$ across-role ratio. Structural; no matcher can move it.

B.4 Reproducing the captured numbers

- Compile with the build line in §9.
- Run from src/ or repo root.
- Pin sample_ward.json to the 2026-04-29 snapshot.

B.5 Mini-glossary

- **Acuity:** per-patient urgency tier (CRIT, HIGH, MED, LOW); smaller is more urgent.
- **Priority list:** list of ints compared left-to-right; smaller is more urgent on every dimension.
- **Makespan:** total schedule length.
- **Spread selection:** pick frontier points that span all three score axes, not just one corner.